PROJECT REPORT

IoT-Based Smart Battery Monitoring System

Using NodeMCU ESP8266, INA219 Current/Voltage Sensor,
0.96-inch SSD1306 OLED and a Wi-Fi Web Dashboard

Subject : Advance Microcontroller (BE05000351)

Submitted by:

Paghdar Manthan (240280111083)

Parkhiya Krish (240280111089)

Parmar Pritesh (240280111092)

Programme: B.E. Electronics & Communication Engineering

Institute: L.D. College of Engineering

Academic Year: 2026–2027

Abstract

This project presents a low-cost smart battery monitoring system built around a NodeMCU ESP8266 microcontroller. An INA219 sensor measures battery-path voltage and current, while the controller calculates electrical power and an estimated state-of-charge percentage for a single-cell 3.7 V nominal lithium-ion battery. A 0.96-inch 128 × 64 SSD1306 OLED provides local readings. The ESP8266 also hosts a responsive web dashboard over Wi-Fi, allowing a phone or computer connected to the same network to view live voltage, current magnitude, power, estimated battery level and charge/discharge/idle status. The dashboard plots recent measurement history in the browser. The prototype photographs and dashboard screenshots supplied with the project show the assembled hardware and successful live operation.

Keywords

ESP8266, NodeMCU, INA219, Li-ion battery, OLED, IoT, Wi-Fi, battery monitoring, web dashboard.

1. Introduction

Rechargeable batteries are used in portable electronics, embedded systems, robotics and backup-power applications. Battery voltage alone does not fully describe how a battery is being used; current and power measurements help identify whether energy is being delivered to a load or supplied during charging. This project combines sensing, embedded processing, a local display and a browser-based interface to make these measurements easier to observe.

The ESP8266 provides Wi-Fi connectivity and enough processing capability to read the sensor, refresh the OLED and serve a small web page. Unlike a dashboard that depends on a cloud account, this implementation serves the page locally from the NodeMCU, so the user opens the IP address printed in the Serial Monitor while connected to the same Wi-Fi network.

2. Problem Statement

Basic battery-powered projects often lack a convenient way to observe voltage, current, power and operating direction while the battery is charging or supplying a load. Manual multimeter checks provide only occasional readings and do not naturally create a live history. The project addresses this by creating a compact, continuously updated monitoring unit with both on-device and wireless viewing.

3. Objectives

- Measure battery-path voltage and current using the INA219 sensor.
- Calculate electrical power from voltage and current.
- Estimate the battery level as a percentage using voltage limits for a single-cell Li-ion battery.
- Classify the operating condition as Charging, Discharging or Idle using the sign and magnitude of measured current.
- Show live readings on a 128 × 64 I²C OLED.
- Host a local Wi-Fi dashboard with live values and recent graphs.
- Build a low-cost prototype that can be adapted to small battery-powered systems.

4. Hardware and Software Requirements

Component	Purpose	Notes
NodeMCU ESP8266 (ESP-12E)	Main controller and Wi-Fi web server	USB-serial interface used for programming
INA219 module	Measures bus voltage, shunt voltage and current	I²C sensor; common address is 0x40
0.96-inch SSD1306 OLED	Local live display	128 × 64 pixels; I²C, commonly address 0x3C
Single-cell Li-ion battery	Battery under observation	3.7 V nominal; full-charge voltage commonly about 4.2 V
Perfboard, wires and USB cable	Prototype assembly and programming	Ensure secure connections and correct polarity
Arduino IDE / ESP8266 core	Compile and upload firmware	Install Adafruit INA219, Adafruit GFX and Adafruit SSD1306 libraries
Wi-Fi-enabled phone/computer	View local dashboard	Must be on the same Wi-Fi network as the ESP8266

5. System Architecture

Battery and load/charger current pass through the INA219 current-sense path. The INA219 communicates with the NodeMCU through I²C. The OLED shares the same I²C bus. The ESP8266 processes sensor values, updates the OLED and exposes two HTTP routes: the root route serves the dashboard page, and the /data route returns the latest measurements as JSON. JavaScript in the browser polls /data once per second and redraws the live values and charts.

Battery / load path → INA219 → I²C → NodeMCU ESP8266 → OLED + Wi-Fi HTTP dashboard

6. Circuit Connections

NodeMCU pin	Connect to	Function
3V3	INA219 VCC and OLED VCC (if module supports 3.3 V)	Sensor/display supply
GND	INA219 GND, OLED GND and battery circuit reference	Common ground
D2 / GPIO4	INA219 SDA and OLED SDA	I²C data
D1 / GPIO5	INA219 SCL and OLED SCL	I²C clock
INA219 VIN+	Source side of the measured current path	Current-sense input
INA219 VIN−	Battery/load side of the measured current path	Current-sense output

Important: connect VIN+ and VIN− in series with the current path; do not connect them as if they were ordinary power-supply pins. Follow the polarity and current-path diagram for the exact INA219 breakout board. Do not route charging and load currents through the sensor simultaneously unless the wiring is intentionally designed for that use. The prototype must not be left unattended while charging.

7. Working Principle

7.1 Sensor acquisition

The firmware initializes the I²C bus, SSD1306 OLED and INA219. At approximately one-second intervals it reads bus voltage, shunt voltage and current from the INA219.

7.2 Voltage calculation

The code combines bus voltage and shunt voltage to estimate the voltage at the measured point. The provided sketch includes a 0.12 V correction term; this should be calibrated against a trusted multimeter for the actual module and wiring.

7.3 Power calculation

Electrical power is calculated as P = V × I. Current is measured in milliamperes, so the code divides the product by 1000 to obtain watts. The dashboard displays the magnitude of current and power.

7.4 Battery percentage

The code maps voltage linearly between 3.0 V and 4.3 V, then limits the result to 0–100%. This is a rough voltage-based estimate, not a laboratory-grade state-of-charge measurement; voltage changes with load, rest time, temperature and cell chemistry.

7.5 Charge/discharge status

With the present wiring and library convention, current above +5 mA is labelled Discharging, below −5 mA is labelled Charging, and values between these limits are labelled Idle. If the sensor is wired in the opposite direction, the two active labels must be swapped.

7.6 OLED and browser dashboard

The OLED displays the IP address, voltage, current magnitude, power, battery percentage and status. The browser dashboard fetches JSON data once per second and plots up to five minutes of recent samples in the current browser session.

8. Software Design

1. Initialize Serial at 115200 baud.
2. Start the I²C bus using SDA = D2 and SCL = D1.
3. Initialize the SSD1306 OLED at I²C address 0x3C.
4. Initialize the INA219; stop and report an error if the sensor is not detected.
5. Connect to the configured Wi-Fi network and print the assigned IP address.
6. Start the HTTP server on port 80 and register / and /data routes.
7. Every second, read measurements, calculate power and battery percentage, classify status, update OLED and print readings to Serial.
8. Respond to browser requests with the dashboard HTML or a JSON object containing voltage, current, power, percentage and status.

9. Dashboard Features and Observed Results

The supplied screenshots show the dashboard reporting a connected state and displaying four live measurement cards, a power-flow status panel and four history charts. The example snapshots include approximately 3.77 V, 551.8 mA, 2.08 W and 59% during a discharging condition; another snapshot shows 4.18 V, 0.0 mA, 0.00 W and 90% while idle; a further snapshot shows 4.29 V, 248.7 mA, 1.07 W and 99% while charging. These are the values visible in the supplied screenshots, not guaranteed calibration results. The 4.29 V reading exceeds the usual 4.20 V full-charge voltage for many standard 3.7 V Li-ion cells, so the voltage measurement/correction term and actual battery voltage should be checked with a multimeter, and the cell must not be overcharged.

10. Project Photographs

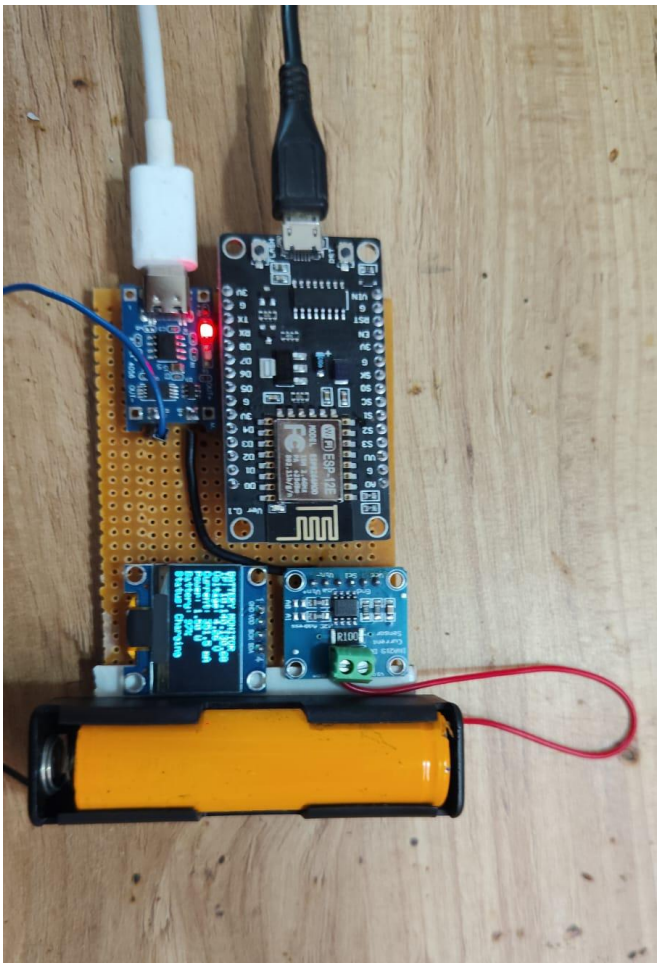


Figure 1. Assembled NodeMCU, INA219, OLED and battery prototype.

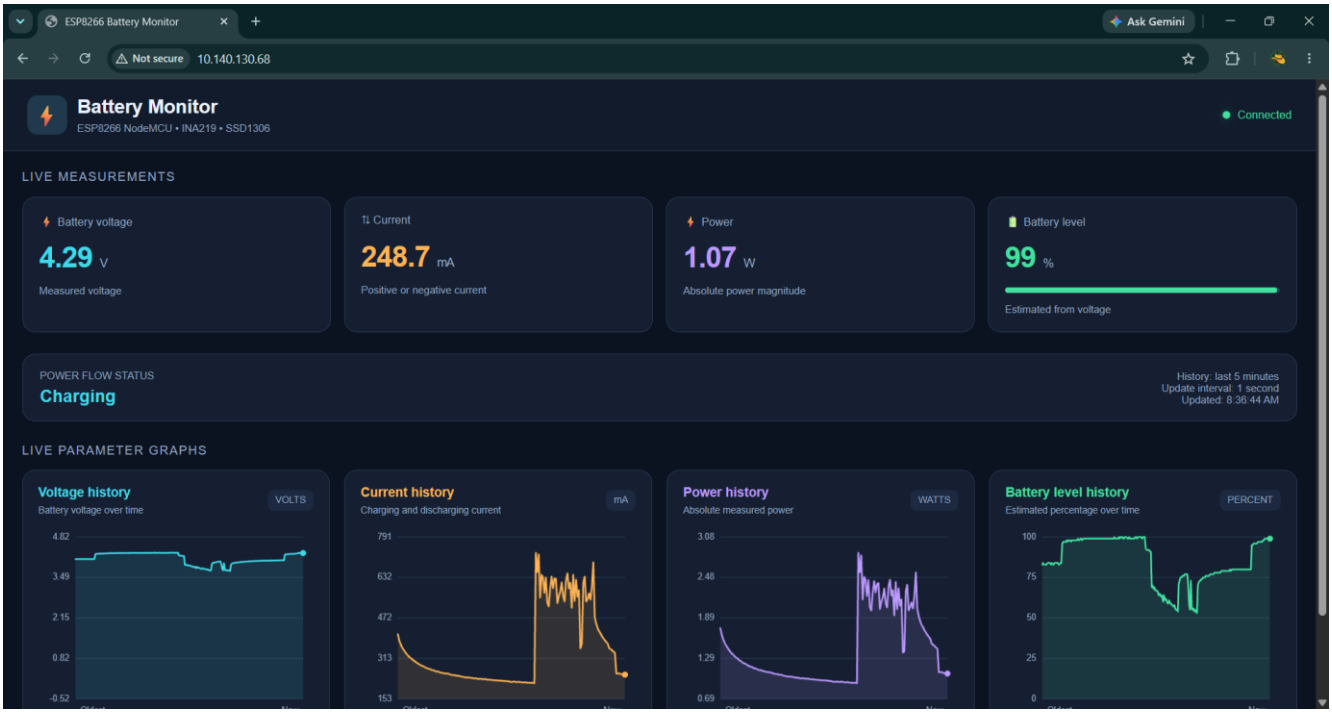


Figure 3. Supplied project dashboard / setup screenshot.

11. Applications

- Battery-powered IoT prototypes and sensor nodes.
- Portable electronics and small embedded systems.
- Robotics and RC projects that need visibility into battery use.
- Educational demonstrations of voltage, current, power and Wi-Fi telemetry.
- Basic testing of battery-powered loads and charging behaviour.

12. Advantages

- Combines electrical sensing, embedded processing, a local display and Wi-Fi monitoring.
- Live values are available without repeatedly connecting a multimeter.
- Browser dashboard works on phones and computers on the same local network.
- Recent trends help reveal changing load current and voltage.
- Uses widely available modules and open-source libraries.

13. Limitations and Safety Considerations

- Battery percentage is estimated from voltage using a linear map; it is not a calibrated state-of-charge gauge.
- INA219 current direction depends on sensor orientation and current-path wiring.
- The local dashboard is not accessible outside the local Wi-Fi network unless additional networking is configured.
- Current and voltage accuracy depend on the breakout board, calibration, wiring resistance and sensor range.
- Use a suitable protected cell/charger. Never exceed the battery manufacturer's charge voltage or current limits.
- Do not power the NodeMCU from the battery directly unless using a suitable regulated supply; the ESP8266 requires a stable 3.3 V rail.

14. Future Improvements

- Calibrate voltage and current against a reliable multimeter and known load.
- Use a chemistry-specific state-of-charge lookup table or coulomb-counting fuel gauge.
- Add low-battery alerts, data logging to a file/cloud service and downloadable CSV history.
- Add overvoltage/undervoltage warnings and a protected battery-management circuit.
- Improve sensor fault reporting and provide a configuration page for Wi-Fi and calibration.
- Use a secure authenticated dashboard if the device is exposed beyond a trusted local network.

15. Mapped Sustainable Development Goals (SDGs)

SDG 7 – Affordable and Clean Energy: supports awareness of energy use and battery performance.

SDG 9 – Industry, Innovation and Infrastructure: demonstrates a practical IoT-based embedded monitoring solution.

SDG 12 – Responsible Consumption and Production: helps users observe battery use and make more informed decisions.

16. Conclusion

The project demonstrates a working prototype for monitoring a single-cell lithium-ion battery with an ESP8266 NodeMCU and INA219 sensor. It provides live voltage, current, power, estimated battery percentage and operating status on both an OLED and a local web dashboard. The dashboard screenshots show the system operating in charging, discharging and idle examples. Before using the prototype for unattended or safety-critical battery operation, the voltage correction should be calibrated and battery protection/charging limits must be verified.

Full Arduino Source Code

Update the Wi-Fi SSID and password for your own network before uploading. it runs a local web server.

```
#include <ESP8266WiFi.h>
#include <ESP8266WebServer.h>
#include <Wire.h>
#include <Adafruit_INA219.h>
#include <Adafruit_GFX.h>
#include <Adafruit_SSD1306.h>

// ------------ WIFI SETTINGS ------------
const char* ssid = "Redmi";
const char* password = "09876543";

// ------------ OLED SETTINGS ------------
#define SCREEN_WIDTH 128
#define SCREEN_HEIGHT 64
#define OLED_RESET -1

Adafruit_SSD1306 display(
  SCREEN_WIDTH, SCREEN_HEIGHT, &Wire, OLED_RESET
);

// ------------ INA219 SENSOR ------------
Adafruit_INA219 ina219;

// ------------ WEB SERVER ------------
ESP8266WebServer server(80);

// ------------ BATTERY SETTINGS ------------
// For a single-cell 3.7V nominal Li-ion battery
const float BATTERY_MIN_VOLTAGE = 3.0;
const float BATTERY_MAX_VOLTAGE = 4.3;

// ------------ SENSOR VALUES ------------
float voltage = 0.0;
float current_mA = 0.0;
float power_W = 0.0;
float powerMagnitudeW = 0.0;
int percentage = 0;
String status = "Starting";

// ------------ DASHBOARD PAGE ------------
const char webpage[] PROGMEM = R"rawliteral(
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>ESP8266 Battery Monitor</title>

<style>
:root {
  --bg: #0b1220;
  --panel: #151f31;
  --panel2: #1c2940;
  --text: #edf4ff;
  --muted: #94a8c5;
  --cyan: #40d9e8;
  --green: #43e0a0;
  --orange: #ffb454;
  --red: #ff7085;
  --border: #2a3a55;
}

* {
  box-sizing: border-box;
}
```

```
body {
  margin: 0;
  background: var(--bg);
  color: var(--text);
  font-family: Arial, Helvetica, sans-serif;
}

header {
  padding: 18px 28px;
  background: #111b2c;
  border-bottom: 1px solid var(--border);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 15px;
  flex-wrap: wrap;
}

.brand {
  display: flex;
  align-items: center;
  gap: 12px;
}

.logo {
  background: #20394c;
  color: var(--cyan);
  width: 46px;
  height: 46px;
  border-radius: 12px;
  display: grid;
  place-items: center;
  font-size: 25px;
}

h1 {
  font-size: 22px;
  margin: 0 0 5px;
}

.subtitle {
  font-size: 12px;
  color: var(--muted);
}

.connection {
  display: flex;
  align-items: center;
  gap: 8px;
  color: var(--green);
  font-size: 13px;
}

.dot {
  width: 9px;
  height: 9px;
  border-radius: 50%;
  background: var(--green);
  box-shadow: 0 0 10px #43e0a066;
}

main {
  padding: 22px;
  max-width: 1800px;
  margin: auto;
}

.section-title {
  font-size: 14px;
```

```
  color: var(--muted);
  text-transform: uppercase;
  letter-spacing: 1.5px;
  margin: 0 0 14px;
}

.metrics {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 15px;
  margin-bottom: 25px;
}

.metric {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 15px;
  padding: 19px;
  min-width: 0;
}

.metric-label {
  color: var(--muted);
  font-size: 13px;
  margin-bottom: 14px;
}

.metric-value {
  font-size: clamp(22px, 2.2vw, 34px);
  font-weight: 700;
  letter-spacing: -1px;
  overflow-wrap: anywhere;
}

.unit {
  font-size: 14px;
  color: var(--muted);
  font-weight: normal;
}

.metric-foot {
  color: var(--muted);
  font-size: 12px;
  margin-top: 12px;
}

.voltage { color: var(--cyan); }
.current { color: var(--orange); }
.power { color: #bd9aff; }
.battery { color: var(--green); }

.progress {
  height: 7px;
  border-radius: 10px;
  background: #2a3950;
  margin-top: 15px;
  overflow: hidden;
}

.progress-fill {
  height: 100%;
  width: 0%;
  background: var(--green);
  transition: width 0.4s;
  border-radius: 10px;
}

.status-panel {
  background: var(--panel);
  border: 1px solid var(--border);
```

```
  border-radius: 15px;
  padding: 17px 20px;
  margin-bottom: 25px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 15px;
  flex-wrap: wrap;
}

.status-label {
  color: var(--muted);
  font-size: 12px;
  margin-bottom: 6px;
}

.status-value {
  color: var(--green);
  font-weight: bold;
  font-size: 20px;
}

.status-value.charging { color: var(--cyan); }
.status-value.discharging { color: var(--orange); }
.status-value.idle { color: var(--muted); }

.status-right {
  color: var(--muted);
  font-size: 12px;
  text-align: right;
}

.graph-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 17px;
}

.graph-card {
  background: var(--panel);
  border: 1px solid var(--border);
  border-radius: 15px;
  padding: 17px 18px 12px;
  min-width: 0;
}

.graph-head {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 10px;
  margin-bottom: 12px;
}

.graph-title {
  font-size: 15px;
  font-weight: bold;
}

.graph-subtitle {
  color: var(--muted);
  font-size: 11px;
  margin-top: 5px;
}

.graph-badge {
  background: var(--panel2);
  color: var(--muted);
  padding: 6px 9px;
  border-radius: 7px;
```

```
  font-size: 11px;
  white-space: nowrap;
}

canvas {
  width: 100%;
  height: 210px;
  display: block;
}

footer {
  text-align: center;
  padding: 22px;
  color: var(--muted);
  font-size: 12px;
}

@media (min-width: 1400px) {
  .graph-grid {
    grid-template-columns: repeat(4, minmax(0, 1fr));
  }
  canvas {
    height: 230px;
  }
}

@media (max-width: 850px) {
  main { padding: 14px; }
  header { padding: 15px; }
  .metrics { grid-template-columns: repeat(2, minmax(0, 1fr)); }
  .graph-grid { grid-template-columns: 1fr; }
  canvas { height: 220px; }
}

@media (max-width: 400px) {
  .metric { padding: 13px; }
  .metric-value { font-size: 23px; }
}
</style>
</head>

<body>

<header>
  <div class="brand">
    <div class="logo">&amp;#9889;</div>
    <div>
      <h1>Battery Monitor</h1>
      <div class="subtitle">ESP8266 NodeMCU &amp;bull; INA219 &amp;bull; SSD1306</div>
    </div>
  </div>

  <div class="connection">
    <span class="dot" id="connectionDot"></span>
    <span id="connectionText">Connecting...</span>
  </div>
</header>

<main>
  <div class="section-title">Live measurements</div>

  <div class="metrics">
    <div class="metric">
      <div class="metric-label">&amp;#9889; Battery voltage</div>
      <div class="metric-value voltage">
        <span id="voltage">--</span>
        <span class="unit">V</span>
      </div>
      <div class="metric-foot">Measured voltage</div>
    </div>
```

```
 <div class="metric">
  <div class="metric-label">&#8645; Current</div>
  <div class="metric-value current">
   <span id="current">--</span>
   <span class="unit">mA</span>
  </div>
  <div class="metric-foot">Positive or negative current</div>
 </div>

 <div class="metric">
  <div class="metric-label">&#9889; Power</div>
  <div class="metric-value power">
   <span id="power">--</span>
   <span class="unit">W</span>
  </div>
  <div class="metric-foot">Absolute power magnitude</div>
 </div>

 <div class="metric">
  <div class="metric-label">&#128267; Battery level</div>
  <div class="metric-value battery">
   <span id="percentage">--</span>
   <span class="unit">%</span>
  </div>
  <div class="progress">
   <div class="progress-fill" id="batteryFill"></div>
  </div>
  <div class="metric-foot">Estimated from voltage</div>
 </div>
</div>

<div class="status-panel">
 <div>
  <div class="status-label">POWER FLOW STATUS</div>
  <div class="status-value" id="status">Starting...</div>
 </div>
 <div class="status-right">
  <div>History: last 5 minutes</div>
  <div>Update interval: 1 second</div>
  <div id="lastUpdate">Waiting for sensor data...</div>
 </div>
</div>

<div class="section-title">Live parameter graphs</div>

<div class="graph-grid">
 <div class="graph-card">
  <div class="graph-head">
   <div>
    <div class="graph-title voltage">Voltage history</div>
    <div class="graph-subtitle">Battery voltage over time</div>
   </div>
   <div class="graph-badge">VOLTS</div>
  </div>
  <canvas id="voltageChart"></canvas>
 </div>

 <div class="graph-card">
  <div class="graph-head">
   <div>
    <div class="graph-title current">Current history</div>
    <div class="graph-subtitle">Charging and discharging current</div>
   </div>
   <div class="graph-badge">mA</div>
  </div>
  <canvas id="currentChart"></canvas>
 </div>

 <div class="graph-card">
```

```
    <div class="graph-head">
     <div>
      <div class="graph-title power">Power history</div>
      <div class="graph-subtitle">Absolute measured power</div>
     </div>
     <div class="graph-badge">WATTS</div>
    </div>
    <canvas id="powerChart"></canvas>
   </div>

   <div class="graph-card">
    <div class="graph-head">
     <div>
      <div class="graph-title battery">Battery level history</div>
      <div class="graph-subtitle">Estimated percentage over time</div>
     </div>
     <div class="graph-badge">PERCENT</div>
    </div>
    <canvas id="percentageChart"></canvas>
   </div>
  </div>
</main>

<footer>
  ESP8266 Local Battery Dashboard &bull; Data is stored in this browser session
</footer>

<script>
const MAX_POINTS = 300; // 5 minutes at 1 sample per second

const historyData = {
  voltage: [],
  current: [],
  power: [],
  percentage: [],
  time: []
};

const charts = [
  {
    id: "voltageChart",
    key: "voltage",
    color: "#40d9e8",
    fill: "rgba(64,217,232,0.12)",
    unit: "V"
  },
  {
    id: "currentChart",
    key: "current",
    color: "#ffb454",
    fill: "rgba(255,180,84,0.12)",
    unit: "mA"
  },
  {
    id: "powerChart",
    key: "power",
    color: "#bd9aff",
    fill: "rgba(189,154,255,0.12)",
    unit: "W"
  },
  {
    id: "percentageChart",
    key: "percentage",
    color: "#43e0a0",
    fill: "rgba(67,224,160,0.12)",
    unit: "%"
  }
];

function drawChart(config) {
```

```
const canvas = document.getElementById(config.id);
if (!canvas) return;

const rect = canvas.getBoundingClientRect();
if (rect.width < 10) return;

const dpr = window.devicePixelRatio || 1;
const width = rect.width;
const height = rect.height;

canvas.width = Math.round(width * dpr);
canvas.height = Math.round(height * dpr);

const ctx = canvas.getContext("2d");
ctx.setTransform(dpr, 0, 0, dpr, 0, 0);
ctx.clearRect(0, 0, width, height);

const left = 43;
const right = 12;
const top = 13;
const bottom = 29;
const plotW = width - left - right;
const plotH = height - top - bottom;

if (plotW <= 0 || plotH <= 0) return;

const values = historyData[config.key];

let minValue = 0;
let maxValue = 1;

if (values.length > 0) {
  minValue = Math.min(...values);
  maxValue = Math.max(...values);

  if (config.key === "percentage") {
    minValue = 0;
    maxValue = 100;
  } else if (config.key === "voltage") {
    minValue = Math.min(minValue, 0);
    maxValue = Math.max(maxValue, 1);
  }

  if (maxValue === minValue) {
    maxValue += 1;
    minValue -= 1;
    if (config.key === "percentage") {
      minValue = 0;
      maxValue = 100;
    }
  }

  const padding = (maxValue - minValue) * 0.12;
  if (config.key !== "percentage") {
    minValue -= padding;
    maxValue += padding;
  }
}

// Horizontal grid lines and Y labels
ctx.font = "10px Arial";
ctx.textAlign = "right";
ctx.textBaseline = "middle";

for (let i = 0; i <= 4; i++) {
  const y = top + plotH * i / 4;
  const val = maxValue - (maxValue - minValue) * i / 4;

  ctx.strokeStyle = "#293951";
  ctx.lineWidth = 1;
```

```
  ctx.beginPath();
  ctx.moveTo(left, y);
  ctx.lineTo(left + plotW, y);
  ctx.stroke();

  ctx.fillStyle = "#94a8c5";
  ctx.fillText(val.toFixed(
    config.key === "percentage" ? 0 :
    config.key === "current" ? 0 : 2
  ), left - 7, y);
}

// X-axis labels
ctx.textAlign = "center";
ctx.textBaseline = "top";
ctx.fillStyle = "#94a8c5";

ctx.fillText("Oldest", left + 20, top + plotH + 9);
ctx.fillText("Now", left + plotW - 15, top + plotH + 9);

if (values.length === 0) {
  ctx.textAlign = "center";
  ctx.textBaseline = "middle";
  ctx.fillStyle = "#94a8c5";
  ctx.fillText("Waiting for measurements...",
    left + plotW / 2, top + plotH / 2);
  return;
}

const points = values.map((value, index) => {
  const x = values.length === 1
    ? left + plotW
    : left + (index / (values.length - 1)) * plotW;

  const y = top + ((maxValue - value) /
    (maxValue - minValue)) * plotH;

  return { x, y };
});

// Filled area
if (points.length > 1) {
  ctx.beginPath();
  ctx.moveTo(points[0].x, top + plotH);
  points.forEach(p => ctx.lineTo(p.x, p.y));
  ctx.lineTo(points[points.length - 1].x, top + plotH);
  ctx.closePath();
  ctx.fillStyle = config.fill;
  ctx.fill();
}

// Line
ctx.beginPath();
points.forEach((p, i) => {
  if (i === 0) ctx.moveTo(p.x, p.y);
  else ctx.lineTo(p.x, p.y);
});

ctx.strokeStyle = config.color;
ctx.lineWidth = 2;
ctx.lineJoin = "round";
ctx.lineCap = "round";
ctx.stroke();

// Latest point
const last = points[points.length - 1];
ctx.beginPath();
ctx.arc(last.x, last.y, 3.5, 0, Math.PI * 2);
ctx.fillStyle = config.color;
ctx.fill();
```

```
}

function drawAllCharts() {
  charts.forEach(drawChart);
}

function setConnection(connected) {
  document.getElementById("connectionText").textContent =
    connected ? "Connected" : "Reconnecting...";

  document.getElementById("connectionDot").style.background =
    connected ? "#43e0a0" : "#ff7085";

  document.getElementById("connectionDot").style.boxShadow =
    connected ? "0 0 10px #43e0a066" : "0 0 10px #ff708566";
}

async function updateDashboard() {
  try {
    const response = await fetch("/data?t=" + Date.now(), {
      cache: "no-store"
    });

    if (!response.ok) throw new Error("HTTP error");

    const data = await response.json();

    document.getElementById("voltage").textContent =
      Number(data.voltage).toFixed(2);

    // Show current magnitude as positive for both charging and discharging.
    document.getElementById("current").textContent =
      Math.abs(Number(data.current)).toFixed(1);

    document.getElementById("power").textContent =
      Number(data.power).toFixed(2);

    document.getElementById("percentage").textContent =
      data.percentage;

    document.getElementById("batteryFill").style.width =
      Math.max(0, Math.min(100, data.percentage)) + "%";

    const statusElement = document.getElementById("status");
    statusElement.textContent = data.status;
    statusElement.className = "status-value";

    const normalizedStatus = String(data.status).toLowerCase();

    if (normalizedStatus === "charging") {
      statusElement.classList.add("charging");
    } else if (normalizedStatus === "discharging") {
      statusElement.classList.add("discharging");
    } else {
      statusElement.classList.add("idle");
    }

    const now = new Date();

    // Voltage and battery percentage continue recording every second.
    historyData.voltage.push(Number(data.voltage));
    historyData.percentage.push(Number(data.percentage));

    // Freeze current and power graph histories while the battery is idle.
    // Resume recording when current is outside the +/-5 mA idle range.
    const measuredCurrent = Number(data.current);
    if (Math.abs(measuredCurrent) > 5) {
      // Plot current magnitude as positive for both charging and discharging.
      historyData.current.push(Math.abs(measuredCurrent));
      historyData.power.push(Number(data.power));
```

```
    }

    historyData.time.push(now.getTime());

    Object.keys(historyData).forEach(key => {
      if (historyData[key].length > MAX_POINTS) {
        historyData[key].shift();
      }
    });

    document.getElementById("lastUpdate").textContent =
      "Updated: " + now.toLocaleTimeString();

    setConnection(true);
    drawAllCharts();

  } catch (error) {
    setConnection(false);
  }
}

window.addEventListener("resize", drawAllCharts);

updateDashboard();
setInterval(updateDashboard, 1000);
</script>

</body>
</html>
)rawliteral";

// ------------ WEBPAGE HANDLER ------------
void handleRoot() {
  server.send_P(200, "text/html", webpage);
}

// ------------ JSON DATA HANDLER ------------
void handleData() {
  String json = "{";

  json += "\"voltage\":";
  json += String(voltage, 3);

  json += ",\"current\":";
  json += String(current_mA, 2);

  json += ",\"power\":";
  json += String(powerMagnitudeW, 3);

  json += ",\"percentage\":";
  json += String(percentage);

  json += ",\"status\":\"";
  json += status;
  json += "\"}";

  server.sendHeader("Cache-Control", "no-store");
  server.send(200, "application/json", json);
}

// ------------ OLED UPDATE ------------
void updateOLED() {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);

  display.setCursor(0, 0);
 display.println("BATTERY MONITOR");

 display.setCursor(0, 8);
```

```
 display.print("IP: ");

if (WiFi.status() == WL_CONNECTED) {
  display.println(WiFi.localIP());
} else {
  display.println("Not connected");
}

  display.setCursor(0, 16);
  display.print("Voltage: ");
  display.print(voltage, 2);
  display.println(" V");

  display.setCursor(0, 26);
  display.print("Current: ");
  display.print(fabs(current_mA), 1);
  display.println(" mA");

  display.setCursor(0, 36);
  display.print("Power: ");
  display.print(powerMagnitudeW, 2);
  display.println(" W");

  display.setCursor(0, 46);
  display.print("Battery: ");
  display.print(percentage);
  display.println("%");

  display.setCursor(0, 56);
  display.print("Status: ");
  display.print(status);

  display.display();
}

// ------------ READ BATTERY ------------
void readBattery() {
  float busVoltage = ina219.getBusVoltage_V();
  float shuntVoltage_mV = ina219.getShuntVoltage_mV();

  voltage = 0.12+busVoltage + shuntVoltage_mV / 1000.0;

  current_mA = ina219.getCurrent_mA();

  // Signed power for internal calculations
  power_W = voltage * current_mA / 1000.0;

  // Absolute power shown on dashboard
  powerMagnitudeW = fabs(power_W);

  // Linear estimate for one Li-ion cell
  float batteryLevel =
    (voltage - BATTERY_MIN_VOLTAGE) /
    (BATTERY_MAX_VOLTAGE - BATTERY_MIN_VOLTAGE) * 100.0;

  percentage = constrain((int)batteryLevel, 0, 100);

  // Current sign depends on INA219 wiring direction
  if (current_mA > 5.0) {
    status = "Discharging";
  } else if (current_mA < -5.0) {
    status = "Charging";
  } else {
    status = "Idle";
  }
}

// ------------ SETUP ------------
void setup() {
  Serial.begin(115200);
```

```
delay(200);

// OLED and INA219 share I2C:
// SDA = D2, SCL = D1
Wire.begin(D2, D1);

// Initialize OLED
if (!display.begin(SSD1306_SWITCHCAPVCC, 0x3C)) {
  Serial.println("OLED initialization failed!");
} else {
  display.clearDisplay();
  display.setTextColor(SSD1306_WHITE);
  display.setTextSize(1);
  display.setCursor(0, 0);
  display.println("Battery Monitor");
  display.println("Starting...");
  display.display();
}

// Initialize INA219
if (!ina219.begin()) {
  Serial.println("INA219 sensor not found!");

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("INA219 NOT FOUND");
  display.println("Check wiring");
  display.display();

  while (true) {
    delay(1000);
  }
}

// Connect Wi-Fi
WiFi.mode(WIFI_STA);
WiFi.begin(ssid, password);

Serial.print("Connecting to WiFi");

unsigned long startTime = millis();

while (WiFi.status() != WL_CONNECTED &&
       millis() - startTime < 20000) {
  delay(500);
  Serial.print(".");
}

Serial.println();

if (WiFi.status() == WL_CONNECTED) {
  Serial.println("WiFi connected!");
  Serial.print("Open dashboard at: http://");
  Serial.println(WiFi.localIP());

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("WiFi Connected!");
  display.print("IP: ");
  display.println(WiFi.localIP());
  display.display();
} else {
  Serial.println("WiFi connection failed!");

  display.clearDisplay();
  display.setCursor(0, 0);
  display.println("WiFi Failed");
  display.println("Check credentials");
  display.display();
}
```

```
  // Register web routes
  server.on("/", HTTP_GET, handleRoot);
  server.on("/data", HTTP_GET, handleData);

  server.begin();
  Serial.println("Web server started");
}

// ------------ MAIN LOOP ------------
void loop() {
  server.handleClient();

  static unsigned long lastRead = 0;

  if (millis() - lastRead >= 1000) {
    lastRead = millis();

    readBattery();
    updateOLED();

    Serial.print("Voltage: ");
    Serial.print(voltage, 2);
    Serial.print(" V | Current: ");
    Serial.print(current_mA, 1);
    Serial.print(" mA | Power: ");
    Serial.print(powerMagnitudeW, 2);
    Serial.print(" W | Battery: ");
    Serial.print(percentage);
    Serial.print("% | Status: ");
    Serial.println(status);
  }

  yield();
}
```